Assignment 4: Sharding of the Developed Application

CheckInOut - Hostel Management System

Group CIO Mohit Kamlesh Panchal (23110208)

CS 432: Databases, Semester II (2025-2026) Instructor: Dr. Yogesh K. Meena IIT Gandhinagar

---

GitHub Repository: https://github.com/rayvego/databases-assignment-4 **Video Demonstration:** https://drive.google.com/file/d/1dlr2leWXm-oxU6HbQ1joHdVpcveVASYd/view?usp=sharing

---

1. Shard Key Selection & Justification

Chosen Shard Key: student_id

Our application is a hostel management system. The central entity is the student: room allocations, gate passes, fee payments, visitor logs, and maintenance requests are all anchored to individual students. We chose student_id as our shard key.

Justification Against the Three Criteria

High Cardinality. student_id is the primary key of the student table. Every student receives a unique integer ID at enrollment, and the value scales linearly with the size of the student body. No two students share an ID. There are no low-cardinality columns like gender (3 values) or status (3-4 values) that would cluster records unevenly. In our current dataset of 35 students, the IDs span a range with no duplicates, and as the system grows to thousands of students the cardinality remains perfect.

Query Alignment. Nearly every hot-path API query in our system filters by student_id. The following endpoints use it in their WHERE clause:

- GET /api/allocations - look up a student's room allocation
- GET /api/gatepasses - retrieve a student's gate pass history
- GET /api/fees - check a student's payment records
- GET /api/maintenance - view maintenance requests filed by a student
- GET /api/students/:id - fetch a student's profile

By sharding on student_id, single-student lookups are resolved by hitting exactly one shard. The router computes student_id % 3 in O(1) and queries only that shard's tables.

Stability. A student_id is assigned once at enrollment and never changes for the lifetime of the record. It is not subject to updates like status (which changes when a gate pass is approved) or current_occupancy (which changes on every check-in/check-out). This eliminates the cost and complexity of moving records between shards after insertion.

Partitioning Strategy: Hash-Based

We use hash-based partitioning: shard_id = student_id % 3.

We considered three strategies:

Strategy	Pros	Cons
Range-Based (IDs 1-100 → Shard 0, 101-200 → Shard 1)	Simple, supports range queries on ID	New students always go to the highest shard, creates temporal skew. Shard N becomes a hot spot.
Directory-Based (lookup table maps keys to shards)	Flexible redistribution, easy rebalancing	Adds a dependency: every query must first consult the directory. Single point of failure.
Hash-Based (student_id % 3)	Even distribution, O(1) routing, no extra lookup	Range queries on student_id still require fan-out. Rebalancing requires rehashing.

We chose hash-based because it guarantees even data distribution without a lookup table. The modulo operation is trivial to compute, adds zero overhead to request routing, and requires no additional infrastructure. The downside is that range queries must fan out to all shards, but this is acceptable because our most common queries are single-student lookups, not range scans.

Expected Data Distribution and Skew Risk

With modulo hashing on sequential integer IDs, records distribute as evenly as mathematically possible: the maximum difference between any two shard sizes is 1 record. Our actual migration results confirm this for the student table:

Shard	Students	Allocations	Gate Passes	Fee Payments	Visit Logs	Maintenance
Shard 0	12	16	8	6	4	203
Shard 1	11	7	2	4	3	3
Shard 2	12	7	3	5	3	4
Total	**35**	**30**	**13**	**15**	**10**	**210**

The student table is nearly perfectly balanced (12, 11, 12). The allocation table shows more variation (16, 7, 7) because students in Shard 0 happen to have more allocations. This is workload skew, not placement skew. The data placement itself is even; the difference is that certain students generate more records than others (e.g., a student with multiple room changes).

The maintenance_request table shows extreme skew (203, 3, 4) because it shards on reported_by (a member_id), and the bulk of the seed data for maintenance requests uses a small set of reported_by values that happen to hash to Shard 0. In a production system with real user-generated data this would normalize. This skew does not affect correctness, only that Shard 0 handles more maintenance queries than the others.

---

2. Data Partitioning

Shard Simulation Approach

Note on Docker deployment: The assignment specification mentions using Docker instances provided by the course staff for shard deployment. However, these instances are only accessible on the IIT Gandhinagar campus network. Since I am off campus until after the submission deadline, I contacted TA Ramanand and received confirmation that simulating sharding on my own machine using separate database instances is acceptable:

> *"In assignment 4 of track 1 of databases course, the shards of the database that you've hosted are only available on the iitgn network and I'm not on campus until 20th which is two days later than the assignment submission deadline. Would it be fine if I used separate database instances to simulate the sharding or do you recommend any other solution?"*
>
> - TA Ramanand: *"You can simulate on your own machine."*

Accordingly, we simulate shard isolation using separate SQLite database files on the local machine, which provides full logical isolation (no shared tables, no cross-database queries, no shared lock files) without requiring Docker or campus network access.

We simulated three physical shard nodes using three separate SQLite database files:

```
shard_0.db    - Shard 0
shard_1.db    - Shard 1
shard_2.db    - Shard 2
```

Each file is a fully independent SQLite database with its own journal, indexes, and connection pool. They run on the same physical machine and share the same disk, but they have no shared state: no shared tables, no cross-database queries, no shared lock files. This simulates the isolation of physically separate database servers. In production, each .db file would live on a different machine connected over a network.

The main application database (sqlite.db) continues to hold reference data (members, staff, rooms, hostel blocks, visitors, users, audit logs) that is not sharded.

Shard Tables Created

Each shard database contains 7 tables - 6 sharded data tables plus a global_id_sequence table:

Table	Columns	Shard Key
shard_X_student	student_id (PK), enrollment_no, course, batch_year, guardian_name, guardian_contact	student_id
shard_X_allocation	allocation_id (PK), student_id, room_id, check_in_date, check_out_date, status	student_id
shard_X_gate_pass	pass_id (PK), student_id, out_time, expected_in_time, actual_in_time, reason, status, approver_id	student_id
shard_X_fee_payment	payment_id (PK), student_id, amount, payment_date, payment_type, transaction_id, status	student_id
shard_X_visit_log	visit_id (PK), visitor_id, student_id, check_in_time, check_out_time, purpose	student_id
shard_X_maintenance_request	request_id (PK), room_id, reported_by, title, description, priority, status, reported_date, resolved_date, resolved_by	reported_by
global_id_sequence	table_name (PK), next_id	N/A

Indexes created per shard:

- idx_shard_X_allocation_student_id on shard_X_allocation(student_id)
- idx_shard_X_allocation_status on shard_X_allocation(status)
- idx_shard_X_gate_pass_student_id on shard_X_gate_pass(student_id)
- idx_shard_X_gate_pass_status on shard_X_gate_pass(status)
- idx_shard_X_maintenance_room_id on shard_X_maintenance_request(room_id)
- idx_shard_X_maintenance_status on shard_X_maintenance_request(status)
- idx_shard_X_fee_student_id on shard_X_fee_payment(student_id)

Tables Not Sharded

The following tables remain in the main sqlite.db as global reference data:

Table	Reason Not Sharded
member	Base entity for all users. Small (~hundreds of rows). Referenced by FK from every sharded table.
staff	Subset of member. Very small (~10 rows). Referenced by gate_pass.approver_id.
users	Auth table. Small. Needed on every request for session validation.
hostel_block	Reference data (~3-10 rows). Never queried per-student.
room	Reference data (~200 rows). Referenced by allocation.room_id. Needs atomic occupancy updates across all allocations.
visitor	Not tied to student_id. Small.
audit_log	System-wide log. Does not have student_id. Written to on every operation.

Data Migration

The migration script (scripts/migrate-data-to-shards.ts) performed the following steps:

1. Opened the main sqlite.db as the source.
2. Opened each shard_X.db as a destination.
3. For each of the 6 sharded tables, read all rows from the source.
4. Computed shard_id = student_id % 3 (or reported_by % 3 for maintenance).
5. Grouped rows by their target shard.
6. Inserted each group into the corresponding shard_X_<table> using a batched transaction per shard.
7. Verified that source row count equals the sum of shard row counts for every table.

The original tables in sqlite.db were not deleted. They remain as backup. After migration, all application writes target the shard databases directly.

Verification Results

The verification script (scripts/verify-sharding.ts) ran four checks per table:

1. **No data loss** - source count equals sum of shard counts. All 6 tables passed.
2. **No duplication** - no primary key appears in more than one shard. All 6 tables passed.
3. **Routing correctness** - for 10 random records per table, the record was found in the shard predicted by shard_key % 3. All 60 sampled records were in the correct shard.
4. **Referential integrity** - every allocation.student_id in a shard has a matching student_id in the same shard's student table. 0 orphan records found.

Full output:

```
✅ PASS | student - No data loss (Source: 35, Shards total: 35)
✅ PASS | student - No duplication across shards
✅ PASS | student - Routing correctness (10 sampled records all in correct shard)
✅ PASS | allocation - No data loss (Source: 30, Shards total: 30)
✅ PASS | allocation - No duplication across shards
✅ PASS | allocation - Routing correctness (10 sampled records all in correct shard)
✅ PASS | gate_pass - No data loss (Source: 13, Shards total: 13)
✅ PASS | gate_pass - No duplication across shards
✅ PASS | gate_pass - Routing correctness (10 sampled records all in correct shard)
✅ PASS | fee_payment - No data loss (Source: 15, Shards total: 15)
✅ PASS | fee_payment - No duplication across shards
✅ PASS | fee_payment - Routing correctness (10 sampled records all in correct shard)
✅ PASS | visit_log - No data loss (Source: 10, Shards total: 10)
✅ PASS | visit_log - No duplication across shards
✅ PASS | visit_log - Routing correctness (10 sampled records all in correct shard)
✅ PASS | maintenance_request - No data loss (Source: 210, Shards total: 210)
✅ PASS | maintenance_request - No duplication across shards
✅ PASS | maintenance_request - Routing correctness (10 sampled records all in correct shard)
✅ PASS | allocation → student - Referential integrity (All allocation student_ids exist in same shard)
```

---

3. Query Routing

How the Shard Router Works

The shard router (lib/shard-router.ts) is a pure module with no side effects. It provides five functions:

Function	Purpose
getShardId(studentId)	Returns studentId % 3
getShardDb(shardId)	Opens (or returns cached) connection to shard_X.db
getAllShardDbs()	Returns array of all 3 shard connections
routeToShard(studentId)	Returns { db, shardId } for direct routing
getNumShards()	Returns 3

Shard DB connections are opened lazily on first access and cached for the lifetime of the Node.js process. Each connection uses WAL journal mode for concurrent read performance.

Request Flow

```
HTTP Request
    │
    ▼
API Route Handler
    │
    ▼
Extract shard key from request
  - student_id (from URL param or request body)
  - reported_by (for maintenance requests)
    │
    ▼
getShardId(key) → computes key % 3
    │
```

```
    ▼
getShardDb(shardId) → returns shard_X.db connection
    │
    ▼
Execute raw SQL query on shard_X_<table>
    │
    ▼
(If needed) JOIN with main DB for global reference data
    │
    ▼
Return JSON response
```

Lookup Queries (Single Shard)

For endpoints that fetch by student_id, the router computes the shard and queries only that shard:

- GET /api/students/:id - shard_id = id % 3, queries shard_X_student, JOINs with member in main DB.
- PUT /api/students/:id - same routing, executes UPDATE on the correct shard.
- DELETE /api/students/:id - same routing, deletes from all 6 sharded tables within that shard (cascade), then deletes from main DB tables (member, users, audit_log).

For lookups by a non-shard-key primary key (e.g., GET /api/allocations/:allocationId), the application does not know which shard holds the record. It fans out to all 3 shards, queries each for the matching ID, and returns the first result found.

Insert Operations

Inserts are routed by extracting the shard key from the request body:

- POST /api/students - inserts into member (main DB) first to get the member_id. Then computes shard_id = member_id % 3 and inserts into shard_X_student.
- POST /api/allocations - extracts student_id from body, routes to that shard, inserts into shard_X_allocation. The room occupancy increment still happens in the main DB (global room table).
- POST /api/gatepasses - extracts student_id, routes to that shard.
- POST /api/fees - extracts student_id, routes to that shard.
- POST /api/maintenance - extracts reported_by, computes shard_id = reported_by % 3.

The global_id_sequence table in each shard DB tracks the next available ID for each table, preventing ID collisions across shards. When the application knows the ID before insertion (as with POST /api/students where member_id is already assigned), it uses that ID directly.

Range and List Queries (Fan-Out)

Endpoints that list all records query shards independently, apply filters per-shard, and merge the results in the application layer. When a query parameter matches the shard key (e.g., studentId), the router computes the shard directly and queries only that one. No fan-out:

```
if (studentIdFilter) {
  // Direct routing: compute shard from student_id
  const shardId = getShardId(Number(studentIdFilter));
  const rows = shardDbs[shardId].prepare(`SELECT * FROM shard_${shardId}_allocation`).all();
  for (const row of rows) {
    if (statusFilter && row.status !== statusFilter) continue;
    allAllocations.push(row);
  }
} else {
  // No shard key - fan out to all 3 shards
  for (let i = 0; i < 3; i++) { ... }
}
```

Supported query parameters per endpoint:

Endpoint	Shard Key (direct route)	Additional Filters
GET /api/students	-	?course=
GET /api/allocations	?studentId=	?status=
GET /api/gatepasses	?studentId=	?status=
GET /api/fees	?studentId=	?status=, ?paymentType=
GET /api/maintenance	?reportedBy=	?status=, ?roomId=
GET /api/visitlogs	?studentId=	?visitorId=

For filters on non-shard-key columns (e.g., status = 'Active'), fan-out to all shards is unavoidable since the status value does not determine shard placement.

Cross-Shard Foreign Key Discussion

The hardest problem in sharding is cross-shard foreign key lookups. If Record A in Shard 0 references Record B in Shard 2, resolving that FK requires a network call between shards. This is slow, unreliable, and hard to make transactional.

We avoid this entirely through **colocated sharding**: every table with a student_id FK is sharded on student_id % 3. This guarantees that a student's allocations, gate passes, fee payments, and visit logs all live in the same shard as the student record itself. FK lookups within a shard are local disk reads, with zero cross-shard communication.

Exception - maintenance_request: This table does not have a student_id column. It is sharded on reported_by (a member_id). In practice, most reporters are students, so the distribution mirrors student-based sharding. However, maintenance records for a given student are NOT guaranteed to be colocated with that student's other records. If the reporter is staff, the maintenance record may live in a different shard than the student who requested it. This is an acknowledged limitation of our design.

Exceptions - shard-to-global lookups:

Some sharded tables reference global (unsharded) tables:

- allocation.room_id → room.room_id
- gate_pass.approver_id → staff.staff_id
- maintenance_request.resolved_by → staff.staff_id
- maintenance_request.reported_by → member.member_id

These are shard-to-global lookups. The shard queries the main DB for the referenced record. This is safe because these tables are small, rarely change, and are always available. However, it creates a dependency: if the main DB goes down, every shard-dependent query that needs this data fails. In production, global tables would be replicated to every shard node. We discuss this limitation below.

We deliberately avoided sharding different tables on different keys (e.g., allocation on allocation_id, student on student_id). That would cause every allocation-to-student join to be a cross-shard operation. Colocated sharding is the industry-standard approach for this reason.

---

4. Scalability & Trade-offs Analysis

Horizontal vs. Vertical Scaling

Vertical scaling means upgrading a single database server: more CPU cores, more RAM, faster SSDs. It is operationally simple. No code changes, no routing logic, no distributed systems complexity. But it has a hard ceiling. A single SQLite file cannot exceed the I/O throughput of one disk. A single server cannot exceed its CPU core count. At some point, no more powerful machine exists (or is affordable).

Horizontal scaling (sharding) means adding more database nodes. Each shard handles ~1/N of the total data and query load. With 3 shards:

- **Write throughput** is ~3x for single-student writes (each write hits one shard, and shards process independently).
- **Read throughput** is ~3x for single-student lookups (each lookup hits one shard).

- **Range queries** still hit all 3 shards. They do not benefit from horizontal scaling because the filter is not the shard key.

The trade-off: horizontal scaling adds significant complexity. Every query must be routed. Inserts must specify a shard. Deletes must cascade across tables within a shard. The application code must manage multiple database connections. Operational overhead increases (monitoring 3 databases instead of 1, backup strategies, failure detection).

For our system, sharding is justified because the student table and its dependents (allocations, gate passes, fees) are the largest and most frequently queried tables. At the benchmark scale (50,000 allocations, 50,000 fee payments, 100,000 audit logs), a single SQLite file would eventually become a bottleneck under concurrent write load. Sharding distributes that load.

Consistency

Each shard is an independent SQLite database. SQLite guarantees ACID properties within a single database file: every transaction is Atomic, Consistent, Isolated, and Durable. Within a single shard, there is no consistency problem.

The problem arises with **cross-shard operations**. Consider deleting a student:

1. Delete from shard_X_student (Shard 0) ✅
2. Delete from shard_X_allocation (Shard 0) ✅
3. Delete from shard_X_gate_pass (Shard 0) ✅
4. Delete from member in main DB ✅

If step 1 succeeds but step 4 fails (main DB is unreachable), the student record is gone from the shard but the member record still exists in the main DB. The system is in an inconsistent state.

We have no distributed transaction coordinator (no two-phase commit, no Saga pattern). Each shard commits independently. This means:

- **Single-shard operations are always consistent.**
- **Multi-shard or cross-database operations are eventually consistent at best, inconsistent at worst.**

Additionally, the global reference tables (room, member, staff) are shared. If the main DB has an uncommitted write to the room table (e.g., a room's current_occupancy was incremented but not yet committed), a shard querying that table may read stale data. In our current implementation, this is mitigated because the main DB uses SQLite's default isolation. But under high concurrent load, this becomes a real risk.

In production, a distributed database would use 2PC (two-phase commit) or eventual consistency with conflict resolution. We do not have the infrastructure for either.

Availability

If one shard goes down (e.g., shard_1.db is corrupted, the file is locked, or the disk is full):

- Students with student_id % 3 == 1 cannot be looked up, cannot check in, cannot get gate passes.
- The other 2/3 of the system (Shard 0 and Shard 2) continue functioning normally.
- Range queries (GET /api/students, GET /api/allocations) will either fail entirely (if our code throws on a shard connection error) or return partial results (2/3 of the data).

This is a partial degradation, better than total system failure (which is what happens with vertical scaling and a single database crash), but worse than a replicated setup where a standby takes over.

If the **main DB** goes down, the impact is catastrophic:

- Auth fails (no users table).
- Student profile lookups fail (no member table for the JOIN).
- Room allocation fails (no room table for occupancy checks).
- All shards are partially functional. They can serve queries that do not need global data, but this is almost nothing.

The main DB is a single point of failure. In production, it would be replicated or replaced with a highly-available database (PostgreSQL with hot standby, etc.).

Partition Tolerance (CAP Theorem)

The CAP theorem states that a distributed system can guarantee at most two of: Consistency, Availability, and Partition Tolerance.

Our system is **partition-tolerant** by design. Shards operate independently, and a network partition between the application and a shard does not crash the other shards.

For the remaining two, our behavior depends on the operation type:

Single-shard lookups: CP (Consistency + Partition Tolerance). If the target shard is unreachable, the query returns an error rather than serving stale data from another shard. We prefer correctness over availability for individual student records.

Range queries: configurable AP or CP. If one shard is down during a list-all query, we could:

- (CP) Fail the entire request and return an error.
- (AP) Return partial results from the reachable shards with a flag indicating which shards responded.

Our current implementation is CP. If any shard throws, the request fails. The AP behavior would be a simple code change: wrap each shard query in a try-catch and collect errors alongside results.

Network partition between app and shard: The application detects the failed connection (SQLite throws on open/query). It can either:

1. Return an HTTP 503 with the specific shard ID that failed (useful for debugging).
2. Return partial results with a warning header (e.g., X-Shard-Status: shard_1_unreachable).

We chose option 1 for single-shard lookups (fail fast) and would choose option 2 for range queries in a production system.

---

5. Observations and Limitations

Global reference tables are a single point of failure. All shards depend on the main sqlite.db for member data, room data, staff data, and authentication. If the main DB is unreachable, every shard-dependent query fails. The solution is to replicate global tables to each shard, but this introduces its own consistency problem (keeping replicas in sync).

No distributed transaction support. Cross-shard deletes (deleting a student cascades to their allocations, gate passes, fees, etc.) are best-effort. If one deletion succeeds and another fails, the system is left in an inconsistent state. Production systems use 2PC or compensating transactions (Sagas). We implement neither.

Shard rebalancing is not implemented. Adding a 4th shard would require rehashing every record (student_id % 4 instead of % 3) and moving approximately 25% of all records to the new shard. During rebalancing, the system would need to serve reads from both old and new shard assignments. This is operationally complex and not implemented.

Hash-based sharding does not optimize range queries on student_id. A query like "find all students with IDs between 100 and 200" must still fan out to all 3 shards, because the modulo hash function does not preserve ordering. A range-based sharding strategy (IDs 1–1000 → Shard 0, 1001–2000 → Shard 1) would optimize this but would create temporal skew. We chose even distribution over range query optimization.

SQLite file-based sharding simulates isolation but shares the same disk. All three .db files live on the same machine, use the same disk I/O bandwidth, and share the same CPU. True production sharding would distribute shards across separate machines with independent disks, networks, and power supplies. Our setup simulates logical isolation but not physical isolation.

The global_id_sequence table is actively used by all sharded table inserts. The table was created and populated during migration with correct max-IDs for each table. Every POST handler for sharded tables (allocations, gate passes, fee payments, visit logs, maintenance requests) calls getNextGlobalId(tableName) before inserting. This allocates the next ID from the global_id_sequence table in the main sqlite.db, guaranteeing globally unique IDs across all shards. Without this, each shard's auto-increment would produce overlapping IDs independently, causing key collisions when child

records from different shards share the same auto-generated ID. The students POST handler does not need it because student_id is derived directly from member_id, already globally unique by the time the shard insert happens.

Maintenance request skew. The maintenance_request table shards on reported_by rather than student_id. In our seed data, 203 of 210 records hash to Shard 0. This is extreme skew and demonstrates a real risk of sharding on non-primary keys. In production, we would investigate whether reported_by values are more evenly distributed in real usage, or whether a different sharding strategy (or leaving the table unsharded) is preferable.